## Knowledge Units

### MSF-Discrete: Discrete Mathematics

***CS Core:***

1. Sets, relations, functions, cardinality
2. Recursive mathematical definitions
3. Proof techniques (induction, proof by contradiction)
4. Permutations, combinations, counting, pigeonhole principle
5. Modular arithmetic
6. Logic: truth tables, connectives (operators), inference rules, formulas, normal forms, simple predicate logic
7. Graphs: basic definitions
8. Order notation

***Illustrative Learning Outcomes:***

***CS Core:***

1. Sets, Relations, and Functions, Cardinality
    a. Explain with examples the basic terminology of functions, relations, and sets.
    b. Perform the operations associated with sets, functions, and relations.
    c. Relate practical examples to the appropriate set, function, or relation model, and interpret the associated operations and terminology in context.
    d. Calculate the size of a finite set, including making use of the sum and product rules and inclusion-exclusion principle.
    e. Explain the difference between finite, countable, and uncountable sets.
2. Recursive mathematical definitions
    a. Apply recursive definitions of sequences or structures (e.g., Fibonacci numbers, linked lists, parse trees, fractals).
    b. Formulate inductive proofs of statements about recursive definitions.
    c. Solve a variety of basic recurrence relations.
    d. Analyze a problem to determine underlying recurrence relations.
    e. Given a recursive/iterative code snippet, describe its underlying recurrence relation, hypothesize a closed form for the recurrence relation, and prove the hypothesis correct (usually, using induction).
3. Proof Techniques
    a. Identify the proof technique used in a given proof.
    b. Outline the basic structure of each proof technique (direct proof, proof by contradiction, and induction) described in this unit.
    c. Apply each of the proof techniques (direct proof, proof by contradiction, and induction) correctly in the construction of a sound argument.
    d. Determine which type of proof is best for a given problem.
    e. Explain the parallels between ideas of mathematical and/or structural induction to recursion and recursively defined structures.

f. Explain the relationship between weak and strong induction and give examples of the appropriate use of each.

4. Permutations, combinations, and counting
   a. Apply counting arguments, including sum and product rules, inclusion-exclusion principle, and arithmetic/geometric progressions.
   b. Apply the pigeonhole principle in the context of a formal proof.
   c. Compute permutations and combinations of a set, and interpret the meaning in the context of the specific application.
   d. Map real-world applications to appropriate counting formalisms, such as determining the number of ways to arrange people around a table, subject to constraints on the seating arrangement, or the number of ways to determine certain hands in cards (e.g., a full house).
5. Modular arithmetic
   a. Perform computations involving modular arithmetic.
   b. Explain the notion of the greatest common divisor and apply Euclid's algorithm to compute it.
6. Logic
   a. Convert logical statements from informal language to propositional and predicate logic expressions.
   b. Apply formal methods of symbolic propositional and predicate logic, such as calculating validity of formulae, computing normal forms, or negating a logical statement.
   c. Use the rules of inference to construct proofs in propositional and predicate logic.
   d. Describe how symbolic logic can be used to model real-life situations or applications, including those arising in computing contexts such as software analysis (e.g., program correctness), database queries, and algorithms.
   e. Apply formal logic proofs and/or informal, but rigorous, logical reasoning to real problems, such as predicting the behavior of software or solving problems such as puzzles.
   f. Describe the strengths and limitations of propositional and predicate logic.
   g. Explain what it means for a proof in propositional (or predicate) logic to be valid.
7. Graphs
   a. Illustrate by example the basic terminology of graph theory, and some of the properties and special cases of types of graphs, including trees.
   b. Demonstrate different traversal methods for trees and graphs, including pre-, post-, and in-order traversal of trees, along with breadth-first and depth-first search for graphs.
   c. Model a variety of real-world problems in computer science using appropriate forms of graphs and trees, such as representing a network topology, the organization of a hierarchical file system, or a social network.
   d. Show how concepts from graphs and trees appear in data structures, algorithms, proof techniques (structural induction), and counting.

**KA Core:**

The recommended topics are the same between CS core and KA-core, but with far more hours, the KA-core can cover these topics in depth and might include more computing-related applications.

### MSF-Probability: Probability

***CS Core:***

1. Basic notions: sample spaces, events, probability, conditional probability, Bayes' rule
2. Discrete random variables and distributions
3. Continuous random variables and distributions
4. Expectation, variance, law of large numbers, central limit theorem
5. Conditional distributions and expectation
6. Applications to computing, the difference between probability and statistics (as subjects)

**KA Core:**
The recommended topics are the same between CS core and KA-core, but with far more hours, the KA-core can cover these topics in depth and might include more computing-related applications.

***Illustrative Learning Outcomes:***
***CS Core:***
1. Basic notions: sample spaces, events, probability, conditional probability, Bayes' rule
    a. Translate a prose description of a probabilistic process into a formal setting of sample spaces, outcome probabilities, and events.
    b. Calculate the probability of simple events.
    c. Determine whether two events are independent.
    d. Compute conditional probabilities, including through applying (and explaining) Bayes' Rule.
2. Discrete random variables and distributions
    a. Define the concept of a random variable and indicator random variable.
    b. Determine whether two random variables are independent.
    c. Identify common discrete distributions (e.g., uniform, Bernoulli, binomial, geometric).
3. Continuous random variables and distributions
    a. Identify common continuous distributions (e.g., uniform, normal, exponential).
    b. Calculate probabilities using cumulative density functions.
4. Expectation, variance, law of large numbers, central limit theorem
    a. Define the concept of expectation and variance of a random variable.
    b. Compute the expected value and variance of simple or common discrete/continuous random variables.
    c. Explain the relevance of the law of large numbers and central limit theorem to probability calculations.
5. Conditional distributions and expectation
    a. Explain the distinction between joint, marginal, and conditional distributions.
    b. Compute marginal and conditional distributions from a full distribution, for both discrete and continuous random variables.
    c. Compute conditional expectations for both discrete and continuous random variables.
6. Applications to computing
    a. Describe how probability can be used to model real-life situations or applications, such as predictive text, hash tables, and quantum computation.
    b. Apply probabilistic processes to solving computational problems, such as through randomized algorithms or in security contexts.

### MSF-Statistics: Statistics

***CS Core:***

1. Basic definitions and concepts: populations, samples, measures of central tendency, variance
2. Univariate data: point estimation, confidence intervals

***KA Core:***

3. Multivariate data: estimation, correlation, regression
4. Data transformation: dimension reduction, smoothing
5. Statistical models and algorithms
6. Hypothesis testing

***Illustrative Learning Outcomes:***

***CS Core:***

1. Basic definitions and concepts: populations, samples, measures of central tendency, variance
    a. Create and interpret frequency tables.
    b. Display data graphically and interpret graphs (e.g., histograms).
    c. Recognize, describe, and calculate means, medians, quantiles (location of data).
    d. Recognize, describe, and calculate variances, interquartile ranges (spread of data).
2. Univariate data: point estimation, confidence intervals
    a. Formulate maximum likelihood estimation (in linear-Gaussian settings) as a least-squares problem.
    b. Calculate maximum likelihood estimates.
    c. Calculate maximum a posteriori estimates and make a connection with regularized least squares.
    d. Compute confidence intervals as a measure of uncertainty.

***KA Core:***

3. Sampling, bias, adequacy of samples, Bayesian vs frequentist interpretations
4. Multivariate data: estimation, correlation, regression
    a. Formulate the multivariate maximum likelihood estimation problem as a least-squares problem.
    b. Interpret the geometric properties of maximum likelihood estimates.
    c. Derive and calculate the maximum likelihood solution for linear regression.
    d. Derive and calculate the maximum a posteriori estimates for linear regression.
    e. Implement both maximum likelihood and maximum a posteriori estimates in the context of a polynomial regression problem.
    f. Formulate and understand the concept of data correlation (e.g., in 2D)
5. Data transformation: dimension reduction, smoothing
    a. Formulate and derive Principal Component Analysis (PCA) as a least-squares problem.
    b. Geometrically interpret PCA (when solved as a least-squares problem).
    c. Describe when PCA works well (one can relate back to correlated data).
    d. Geometrically interpret the linear regression solution (maximum likelihood).
6. Statistical models and algorithms
    a. Apply PCA to dimensionality reduction problems.
    b. Describe the tradeoff between compression and reconstruction power.

c. Apply linear regression to curve-fitting problems.
d. Explain the concept of overfitting.
e. Discuss and apply cross-validation in the context of overfitting and model selection (e.g., degree of polynomials in a regression context).

### MSF-Linear: Linear Algebra

***CS Core:***

1. Vectors: definitions, vector operations, geometric interpretation, angles: Matrices: definition, matrix operations, meaning of Ax=b.

***KA Core:***

2. Matrices, matrix-vector equation, geometric interpretation, geometric transformations with matrices
3. Solving equations, row-reduction
4. Linear independence, span, basis
5. Orthogonality, projection, least-squares, orthogonal bases
6. Linear combinations of polynomials, Bezier curves
7. Eigenvectors and eigenvalues
8. Applications to computer science: Principal Components Analysis (PCA), Singular Value Decomposition (SVD), page-rank, graphics

***Illustrative Learning Outcomes:***

***CS Core:***

1. Vectors: definitions, vector operations, geometric interpretation, angles
   a. Describe algebraic and geometric representations of vectors in $R^n$ and their operations, including addition, scalar multiplication, and dot product.
   b. List properties of vectors in $R^{n.}$
   c. Compute angles between vectors in $R^{n.}$

**KA Core:**

2. Matrices, matrix-vector equation, geometric interpretation, geometric transformations with matrices
   a. Perform common matrix operations, such as addition, scalar multiplication, multiplication, and transposition.
   b. Relate a matrix to a homogeneous system of linear equations.
   c. Recognize when two matrices can be multiplied.
   d. Relate various matrix transformations to geometric illustrations.
3. Solving equations, row-reduction
   a. Formulate, solve, apply, and interpret properties of linear systems.
   b. Perform row operations on a matrix.
   c. Relate an augmented matrix to a system of linear equations.
   d. Solve linear systems of equations using the language of matrices.
   e. Translate word problems into linear equations.
   f. Perform Gaussian elimination.
4. Linear independence, span, basis
   a. Define subspace of a vector space.

   b. List examples of subspaces of a vector space.
   c. Recognize and use basic properties of subspaces and vector spaces.
   d. Determine if specific subsets of a vector space are subspaces.
   e. Discuss the existence of a basis of an abstract vector space.
   f. Describe coordinates of a vector relative to a given basis.
   g. Determine a basis for and the dimension of a finite-dimensional space.
   h. Discuss spanning sets for vectors in $R^n$.
   i. Discuss linear independence for vectors in $R^n$.
   j. Define the dimension of a vector space.
5. Orthogonality, projection, least-squares, orthogonal bases
   a. Explain the Gram-Schmidt orthogonalization process.
   b. Define orthogonal projections.
   c. Define orthogonal complements.
   d. Compute the orthogonal projection of a vector onto a subspace, given a basis for the subspace.
   e. Explain how orthogonal projections relate to least square approximations.
6. Linear combinations of polynomials, Bezier curves
   a. Identify polynomials as generalized vectors.
   b. Explain linear combinations of basic polynomials.
   c. Describe orthogonality for polynomials.
   d. Distinguish between basic polynomials and Bernstein polynomials.
   e. Apply Bernstein polynomials to Bezier curves.
7. Eigenvectors and eigenvalues
   a. Find the eigenvalues and eigenvectors of a matrix.
   b. Define eigenvalues and eigenvectors geometrically.
   c. Use characteristic polynomials to compute eigenvalues and eigenvectors.
   d. Use eigenspaces of matrices, when possible, to diagonalize a matrix.
   e. Perform diagonalization of matrices.
   f. Explain the significance of eigenvectors and eigenvalues.
   g. Find the characteristic polynomial of a matrix.
   h. Use eigenvectors to represent a linear transformation with respect to a particularly nice basis.
8. Applications to computer science: PCA, SVD, page-rank, graphics
   a. Explain the geometric properties of PCA.
   b. Relate PCA to dimensionality reduction.
   c. Relate PCA to solving least-squares problems.
   d. Relate PCA to solving eigenvector problems.
   e. Apply PCA to reducing the dimensionality of a high-dimensional dataset (e.g., images).
   f. Explain the page-rank algorithm and understand how it relates to eigenvector problems.
   g. Explain the geometric differences between SVD and PCA.
   h. Apply SVD to a concrete example (e.g., movie rankings).

### MSF-Calculus

***KA Core:***

1. Sequences, series, limits

2. Single-variable derivatives: definition, computation rules (chain rule etc.), derivatives of important functions, applications
3. Single-variable integration: definition, computation rules, integrals of important functions, fundamental theorem of calculus, definite vs indefinite, applications (including in probability)
4. Parametric and polar representations
5. Taylor series
6. Multivariate calculus: partial derivatives, gradient, chain-rule, vector valued functions,
7. Optimization: convexity, global vs local minima, gradient descent, constrained optimization, and Lagrange multipliers.
8. Ordinary Differential Equations (ODEs): definition, Euler method, applications to simulation, Monte Carlo integration
9. CS applications: gradient descent for machine learning, forward and inverse kinematics, applications of calculus to probability

***Note:*** the calculus topics listed above are aligned with computer science goals rather than with traditional calculus courses. For example, multivariate calculus is often a course by itself, but computer science undergraduates only need parts of it for machine learning.

***Illustrative Learning Outcomes:***

***KA Core:***

1. Sequences, series, limits
    a. Explain the difference between infinite sets and sequences.
    b. Explain the formal definition of a limit.
    c. Derive the limit for examples of sequences and series.
    d. Explain convergence and divergence.
    e. Apply L'Hospital's rule and other approaches to resolving limits.
2. Single-variable derivatives: definition, computation rules (chain rule etc.), derivatives of important functions, applications
    a. Explain a derivative in terms of limits.
    b. Explain derivatives as functions.
    c. Perform elementary derivative calculations from limits.
    d. Apply sum, product, and quotient rules.
    e. Work through examples with important functions.
3. Single-variable integration: definition, computation rules, integrals of important functions, fundamental theorem of calculus, definite vs indefinite, applications (including in probability)
    a. Explain the definitions of definite and indefinite integrals.
    b. Apply integration rules to examples with important functions.
    c. Explore the use of the fundamental theorem of calculus.
    d. Apply integration to problems.
4. Parametric and polar representations
    a. Apply parametric representations of important curves.
    b. Apply polar representations.
5. Taylor series
    a. Derive Taylor series for some important functions.
    b. Apply the Taylor series to approximations.

6. Multivariate calculus: partial derivatives, gradient, chain-rule, vector valued functions, applications to optimization, convexity, global vs local minima.
    a. Compute partial derivatives and gradients.
    b. Work through examples with vector-valued functions with gradient notation.
    c. Explain applications to optimization.
7. ODEs: definition, Euler method, applications to simulation
    a. Apply the Euler method to integration.
    b. Apply the Euler method to a single-variable differential equation.
    c. Apply the Euler method to multiple variables in an ODE.

## Professional Dispositions

We focus on dispositions helpful to students learning mathematics.

- ***Growth mindset***. Perhaps the most important of the dispositions, students should be persuaded that anyone can learn mathematics, certainly the subset foundational to CS, and that success is not dependent on innate ability.
- ***Practice mindset***. Students should be educated about the nature of "doing" mathematics and learning through practice with problems as opposed to merely listening or observing demonstrations in the classroom.
- ***Deferred gratification***. Most students are likely to learn at least some mathematics from mathematics departments unfamiliar with computing applications; computing departments should acclimate the students to the notion of waiting to see computing applications. Many of the new growth areas such as AI or quantum computing can serve as motivation.
- ***Persistence***. Student perceptions are often driven by frustration with inability to solve hard problems that they see some peers tackle seemingly effortlessly; computing departments should help promote the notion that eventual success through persistence is what matters.
- ***Skepticism and inquiry***. Students often look for "given formulas" as handed down by experts only to be memorized and used. Yet, a theoretical mindset and, more broadly, a scientific one, should feature skepticism and a curiosity about how formulas are established.

## Mathematics Requirements

The most important topics expected from students entering a computing program typically correspond to pre-calculus courses in high school.

**Required:**

- Algebra and numeracy
    - Numeracy: numbers, operations, types of numbers, fluency with arithmetic, exponent notation, rough orders of magnitude, fractions, and decimals.
    - Algebra: rules of exponents, solving linear or quadratic equations with one or two variables, factoring, algebraic manipulation of expressions with multiple variables.
- Precalculus
    - Coordinate geometry: distances between points, areas of common shapes.
    - Functions: function notation, drawing and interpreting graphs of functions.